



DULCE CÓDIGO S.A. — MISIÓN OBJETOS

Taller 100 % guiado de Java: clases, atributos, métodos y constructor

La misma planta de dulces de café, ahora modelada con objetos

Primer semestre de universidad

5 niveles + jefe final · 600 XP · 4–5 horas

Con protocolo de uso responsable de Inteligencia Artificial

"Cada producto, cada camión y cada pedido debe ser UNA sola cosa en el programa. Los ingenieros llaman a eso objetos. Y Java significa café: estábamos destinados."

CONTENIDO

Si el índice aparece vacío, presione Ctrl + A y luego F9 para actualizarlo.

 CONTENIDO	2
Taller 100% guiado de Java: clases, atributos, métodos y constructor	5
 LA HISTORIA (léela, es parte del taller)	5
 CÓMO FUNCIONA CADA NIVEL	6
Puntos (XP)	6
 TU PUESTO DE TRABAJO: cero instalación	6
 PROTOCOLO DE IA: tu supervisor virtual	7
NIVEL 1 — EL MOLDE DE LOS PRODUCTOS 	8
Clases, atributos y objetos	8
 TE EXPLICO	8
 EJEMPLO PRÁCTICO — Los tres primeros productos del catálogo	9
 PRUEBA Y DESCUBRE (10 XP — <i>anota qué pasó en cada uno</i>)	10
 HAZLO TÚ — La flota de camiones (30 XP)	10
 RETO ADICIONAL (20 XP)	11
 DETECTIVE DE ERRORES #1 (20 XP)	11
 USA LA IA PARA APRENDER (10 XP)	11
 CHECKPOINT — XP: ___/90	12
NIVEL 2 — PRODUCTOS QUE ACTÚAN 	13
Métodos con parámetros (acciones void)	13
 TE EXPLICO	13
 EJEMPLO PRÁCTICO — Recibir y despachar con reglas	14
 PRUEBA Y DESCUBRE (10 XP)	15
 HAZLO TÚ — El camión operativo (30 XP)	15
 RETO ADICIONAL (20 XP)	16
 DETECTIVE DE ERRORES #2 (20 XP)	16
 USA LA IA PARA APRENDER (10 XP)	16
 CHECKPOINT — XP: ___/90	17
NIVEL 3 — PRODUCTOS QUE RESPONDEN 	18
Métodos que devuelven valores (return)	18
 TE EXPLICO	18
 EJEMPLO PRÁCTICO — El producto que responde preguntas	19
 PRUEBA Y DESCUBRE (10 XP)	20
 HAZLO TÚ — El camión que responde (30 XP)	20
 RETO ADICIONAL (20 XP)	21

 DETECTIVE DE ERRORES #3 (20 XP)	21
 USA LA IA PARA APRENDER (10 XP)	22
 CHECKPOINT — XP: ____/90.....	22
NIVEL 4 — LA MÁQUINA DE NACER 	23
El constructor y <code>this</code>	23
 TE EXPLICO	23
 EJEMPLO PRÁCTICO — El registro oficial de productos	24
 PRUEBA Y DESCUBRE (10 XP)	25
 HAZLO TÚ — La flota nace completa (30 XP)	25
 RETO ADICIONAL (20 XP)	26
 DETECTIVE DE ERRORES #4 — el más traicionero del taller (20 XP)	26
 USA LA IA PARA APRENDER (10 XP)	27
 CHECKPOINT — XP: ____/90.....	27
NIVEL 5 — LA PLANTA EN MOVIMIENTO 	28
Objetos que colaboran entre sí.....	28
 TE EXPLICO	28
 EJEMPLO PRÁCTICO — El despacho conversado	29
 PRUEBA Y DESCUBRE (10 XP)	30
 HAZLO TÚ — El pedido completo (30 XP)	31
 RETO ADICIONAL (20 XP)	31
 DETECTIVE DE ERRORES #5 (20 XP).....	31
 USA LA IA PARA APRENDER (10 XP)	32
 CHECKPOINT — XP: ____/90.....	32
 JEFE FINAL — "BODEGA MAESTRA" 	33
Proyecto integrador ·  60–90 min · 150 XP	33
La situación	33
El sistema SIBOM (Sistema de Bodega — Misión Objetos).....	33
Restricciones de calidad (<i>aquí vive la nota</i>)	34
Entregables	34
 ANEXOS	35
ANEXO A — Chuleta: POO en Java en una página	35
ANEXO B — Diccionario del compilador (versión objetos)	35
ANEXO C — Soluciones de los Detectives.....	36
ANEXO D — Rúbrica de evaluación.....	37
ANEXO E — Tablero de XP y Bitácora de IA	38
ANEXO F — Guía para el docente	¡Error! Marcador no definido.
 CIERRE	¡Error! Marcador no definido.

Taller 100% guiado de Java: clases, atributos, métodos y constructor

Nivel: Primer semestre · **Edad sugerida:** 16 años en adelante · **Duración:** 4–5 horas

Temáticas (solo estas): clases y atributos · objetos e instancias · métodos · constructor

Ya sabes (y aquí se usa sin re-explicar): variables, operadores y todas las estructuras de control (`if/else`, `switch`, `while`, `do-while`, `for`)

Lo que NO veremos: `private`/getters, herencia, interfaces, `ArrayList`. Eso viene en el siguiente taller.



LA HISTORIA (léela, es parte del taller)

La planta **Dulce Café S.A.** creció. El sistema que construiste con variables sueltas ya no da abasto: para 3 productos son 12 variables; para los 40 del catálogo nuevo serían **160 variables sincronizadas a mano**. La jefa de operaciones te cita en su oficina:

"El inventario ya no se puede manejar con variables regadas. Necesito que cada producto, cada camión y cada pedido sea una sola cosa en el programa: que viaje completo, que sepa cuidarse solo y que nadie pueda dañarlo por accidente. Los ingenieros llaman a eso objetos. Y hay un detalle que me encanta: el lenguaje que vamos a usar se llama Java... que significa café. Estábamos destinados."

En este taller vas a **modelar la planta**: convertir productos, camiones y despachos en objetos de Java que tienen datos (atributos), saben hacer cosas (métodos) y nacen completos (constructor).

Nivel	Qué construyes en la planta	Tema de Java
1	El molde de los productos	Clases, atributos y objetos (<code>new</code> , el punto)
2	Productos que actúan: recibir y despachar	Métodos con parámetros
3	Productos que responden preguntas	Métodos que devuelven valores (<code>return</code>)
4	La máquina de nacer productos	Constructor y <code>this</code>
5	La planta en movimiento: objetos que colaboran	Todo junto: métodos entre objetos
	El sistema de bodega completo	Proyecto integrador

CÓMO FUNCIONA CADA NIVEL

Las mismas 8 secciones de siempre, en el mismo orden:

Sección	Ícono	Qué pasa allí
TE EXPLICO		El concepto, claro y corto, con una metáfora de la planta
EJEMPLO PRÁCTICO		Código real que escribes y ejecutas, con su salida exacta
PRUEBA Y DESCUBRE		Cambios pequeños al código para descubrir reglas por ti mismo
HAZLO TÚ		Tu ejercicio del nivel, con pistas escalonadas
RETO ADICIONAL		Para subir la dificultad
DETECTIVE DE ERRORES		Código roto de la planta que debes diagnosticar y reparar
USA LA IA PARA APRENDER		Un ejercicio concreto con IA, con reglas
CHECKPOINT		Autoevaluación en una línea

 **La regla de oro: NO LEAS EL CÓDIGO — ESCRÍBELO.** Los dedos aprenden lo que los ojos solo miran.

Puntos (XP)

Acción	XP		Acción	XP
Ejecutar el ejemplo y los descubrimientos 	10		Detective  resuelto y explicado	20
HAZLO TÚ  funcionando	30		Ejercicio de IA  documentado	10
Reto adicional 	20		Total por nivel	90

5 niveles = 450 XP · Jefe final = 150 XP · GRAN TOTAL = 600 XP

 Auxiliar de bodega: 180 ·  Analista de objetos: 330 ·  Arquitecto de la planta: 450 ·  Gerente de sistemas: 520+ con jefe final aprobado

TU PUESTO DE TRABAJO: cero instalación

1. Entra a ``programiz.com/java-programming/online-compiler`` (o `onlinegdb.com` eligiendo Java).
2. Verás una clase `Main` con un método `main`. **Todo programa Java arranca ahí**; es la puerta de entrada.
3. Presiona **Run** y confirma que imprime algo.

⚠ **La regla del compilador en línea (léela dos veces, te ahorra media hora):** en estos compiladores todo va **en el mismo archivo**, y solo la clase **Main** lleva la palabra **public**. Las clases que tú crees (**Producto**, **Camion**...) se escriben **debajo de Main, SIN `public`**:

```
```java
public class Main {
 public static void main(String[] args) {
 // aquí va tu programa
 }
}

class Producto {
 // aquí va tu clase
}
```
```

Si trabajas en un computador con IntelliJ o NetBeans, cada clase va en su propio archivo (**Producto.java**) y ahí sí todas pueden ser **public**. Ambas formas son correctas; usa la de tu entorno.

Repaso exprés de lo que ya dominas (solo para calibrar la sintaxis Java): variables con tipo (**int cajas = 50;** · **double peso = 6.2;** · **String nombre = "Caramelos";** · **boolean ok = true;**), impresión con **System.out.println(...)**, y todas las estructuras de control con llaves **{ }**. Si algo de esto te suena borroso, repásalo 10 minutos antes de empezar: este taller lo usa desde la primera página.

PROTOCOLO DE IA: tu supervisor virtual

1. **Regla de los 60 segundos:** intenta primero tú, un minuto, con la mente.
2. **Regla del filtro** — todo prompt termina con esta frase (cópiala y guárdala):
 > "Usa **SOLO**: clases con atributos sin modificadores, objetos con **new**, métodos y constructores, más estructuras de control básicas. **NO** uses **private**, **getters/setters**, **herencia**, **interfaces**, **ArrayList**, **static** (salvo el **main**) ni **sobrecarga**. Nivel principiante. Todo en un solo archivo con la clase **Main**."
3. **Regla de la firma:** no entregues nada que no puedas explicar línea por línea.

 Cada consulta se anota en la **Bitácora de IA** (Anexo E).

NIVEL 1 — EL MOLDE DE LOS PRODUCTOS

Clases, atributos y objetos

 45 min · 90 XP

TE EXPLICO

Hoy la ficha de un producto vive regada en variables: `nombreCaramelos`, `stockCaramelos`, `pesoCajaCaramelos`... Tres datos por producto, cuarenta productos: un caos que crece sin control. La solución de Java (y de casi toda la industria) tiene dos piezas:

Una CLASE es un molde: describe qué datos tendrá toda cosa de ese tipo. Un OBJETO es cada cosa concreta fabricada con ese molde. El molde de galletas es uno; las galletas son muchas, y morderle la oreja a una no daña a las demás.

Pieza 1 — Definir la clase (el molde):

```
class Producto {
    String nombre;
    int stockCajas;
    double pesoCajaKg;
    boolean aprobado;
}
```

Léelo así: "todo Producto tendrá un nombre (texto), un stock en cajas (entero), un peso por caja (decimal) y un estado de aprobación (booleano)". Los datos que el molde promete se llaman **atributos**. Fíjate en dos costumbres profesionales: la clase va en **PascalCase** (`Producto`, `CamionRepartidor`) y los atributos en **camelCase** (`stockCajas`, `pesoCajaKg`).

Pieza 2 — Fabricar objetos (las galletas):

```
Producto caramelos = new Producto();
```

| | | | |
|--------------|-------------|---|-----------------|
| Producto | caramelos | = | new Producto(); |
| ↑ | ↑ | | ↑ |
| tipo | nombre de | | "fabricame uno |
| (¡tu clase | la variable | | nuevo con este |
| es un tipo!) | | | molde" |

``new`` fabrica el objeto en la memoria. Sin `new` solo existe el molde. Y una vez fabricado, se trabaja con el **punto**, que se lee como "de": `caramelos.stockCajas` = "el stock de caramelos". Sirve para leer y para escribir.

📌 **Los valores de fábrica de Java:** si no asignas un atributo, Java le pone un valor por defecto según su tipo: `int` → `0`, `double` → `0.0`, `boolean` → `false`, y `String` (o cualquier objeto) → `null`, que significa "aquí no hay nada". Ese `null` te lo vas a encontrar en el 🐛 de hoy... y en toda tu carrera.

📁 EJEMPLO PRÁCTICO — Los tres primeros productos del catálogo

Escribe el archivo completo y ejecútalo:

```
public class Main {
    public static void main(String[] args) {

        // ---- Fabricamos y personalizamos el primer producto ----
        Producto caramelos = new Producto();
        caramelos.nombre = "Caramelos de café";
        caramelos.stockCajas = 50;
        caramelos.pesoCajaKg = 6.2;
        caramelos.aprobado = true;

        // ---- El segundo, con otros datos ----
        Producto arequipe = new Producto();
        arequipe.nombre = "Arequipe de café";
        arequipe.stockCajas = 30;
        arequipe.pesoCajaKg = 4.8;
        arequipe.aprobado = true;

        // ---- El tercero, recién creado: puros valores de fábrica ----
        Producto misterioso = new Producto();

        System.out.println("==== CATÁLOGO =====");
        System.out.println(caramelos.nombre + " | " + caramelos.stockCajas + " cajas | " +
            caramelos.pesoCajaKg + " kg/caja");
        System.out.println(arequipe.nombre + " | " + arequipe.stockCajas + " cajas | " +
            arequipe.pesoCajaKg + " kg/caja");
        System.out.println(misterioso.nombre + " | " + misterioso.stockCajas + " cajas | aprobado:
            " + misterioso.aprobado);

        // ---- Cada objeto es independiente ----
        caramelos.stockCajas = 38; // despachamos 12 cajas de caramelos
        System.out.println("\nTras el despacho:");
        System.out.println("Caramelos: " + caramelos.stockCajas + " | Arequipe: " +
            arequipe.stockCajas);
    }
}

class Producto {
    String nombre;
    int stockCajas;
    double pesoCajaKg;
    boolean aprobado;
}
```

🖥️ En pantalla verás:

```
===== CATÁLOGO =====
Caramelos de café | 50 cajas | 6.2 kg/caja
Arequipe de café | 30 cajas | 4.8 kg/caja
null | 0 cajas | aprobado: false
Tras el despacho:
Caramelos: 38 | Arequipe: 30
```

Dos lecciones a la vista: el producto `misterioso` muestra **los valores de fábrica** (`null`, `0`, `false`) porque nadie lo personalizó; y bajar el stock de caramelos **no tocó** al arequipe — cada objeto tiene **su propia copia** de los atributos. Un molde, muchos objetos, vidas separadas.



PRUEBA Y DESCUBRE (10 XP — anota qué pasó en cada uno)

1. Borra la línea `Producto misterioso = new Producto();` pero deja sus `println`. El compilador reclama: `cannot find symbol`. Sin fabricar, la variable no existe.
2. Ahora prueba esto: `Producto p; p.nombre = "X";` (declarada pero **sin** `new`). Otro reclamo: `variable p might not have been initialized`. **Declarar no es fabricar.**
3. El experimento estrella: escribe `Producto clon = caramelos;` (sin `new`) y luego `clon.stockCajas = 5;`. Imprime `caramelos.stockCajas`. ¿Qué pasó? La variable no guarda el objeto: guarda **la dirección donde vive**. Dos nombres, un solo objeto. ¿Cuántos `new` necesitas para dos productos de verdad?
4. Intenta `caramelos.color = "café";`. El compilador: `cannot find symbol`. Si no está en el molde, ningún objeto puede tenerlo. **El molde manda.**



HAZLO TÚ — La flota de camiones (30 XP)

La planta compró camiones repartidores. Modélos:

1. Crea la clase ``Camion`` con estos atributos (elige bien los tipos): placa, capacidad en kg, carga actual en kg, ciudad asignada y disponible (sí/no).
2. En `main`, fabrica **tres camiones**: dos completamente personalizados y uno solo con placa (el resto, valores de fábrica).
3. Imprime la ficha de los tres: `Placa ABC-123 | Armenia | 1200.0 kg máx | carga: 350.5 kg | disponible: true`.
4. Simula la operación del día **solo con asignaciones**: el camión 1 carga 200 kg más (súmalos a su carga actual), el camión 2 sale de servicio (`disponible = false`), e imprime las fichas actualizadas demostrando que el camión 3 no cambió en nada.

Pistas: 🔑 1) La clase va debajo de `Main`, sin `public`. 🔑 2) "Carga 200 kg más" es `camion1.cargaActualKg = camion1.cargaActualKg + 200;` (o `+=`). 🔑 3) Tres camiones = tres `new`.

RETO ADICIONAL (20 XP)

Usando **solo estructuras de control que ya sabes** y los atributos: recorre mentalmente tus tres camiones e imprime con **if** cuál puede recibir un pedido de 400 kg (cabe si `cargaActual + 400 <= capacidad` y está disponible). Hazlo con tres **if** independientes, uno por camión. En el Nivel 3 descubrirás cómo convertir esa pregunta repetida en algo mucho más elegante.

DETECTIVE DE ERRORES #1 (20 XP)

El practicante modeló el cliente y nada compila:

```
class cliente {
    string nombre;
    int pedidosRealizados;
}

// ...dentro del main:
cliente c1 = cliente();
c1.nombre = "Dulcería El Trébol";
System.out.println(c1.Nombre);
```

Diagnóstico: ① ¿Qué convención rompe `class cliente` y por qué importa aunque compile? ② `string` con minúscula no existe en Java — ¿cómo se escribe el tipo de texto? ③ A la línea del `new` le falta justamente... ¿qué palabra, y qué pasa sin ella? ④ ¿Por qué `c1.Nombre` falla si el atributo "sí existe"? Corrige las cuatro cosas.

USA LA IA PARA APRENDER (10 XP)

Pídele: "Modela en Java la clase Bodega para una planta de dulces, con 5 atributos y sus tipos" (+ el filtro del protocolo). **Audita su respuesta:** ¿los tipos tienen sentido (¿una temperatura como `int` o como `double`)? ¿Respetó camelCase? ¿Se coló algo prohibido (`private`, métodos)? Si se coló, recórtalo tú y anota en la bitácora qué tuvo que salir.

✓ CHECKPOINT — XP: ____/90

☐ Explico clase vs. objeto con el molde de galletas · ☐ Sé que `new` fabrica y que declarar no es fabricar · ☐ Conozco los valores de fábrica (`0`, `0.0`, `false`, `null`) · ☐ Entendí que la variable guarda una dirección (experimento 3)

NIVEL 2 — PRODUCTOS QUE ACTÚAN

Métodos con parámetros (acciones `void`)

 45 min · 90 XP

TE EXPLICO

Tus productos ya tienen datos, pero son fichas inertes: para despachar cajas, el código de afuera les manipula el stock a mano. Eso tiene dos problemas: se repite en todas partes, y nadie valida nada — ayer alguien dejó un stock en -12.

Los atributos son lo que el objeto ES. Los métodos son lo que el objeto SABE HACER. Y la regla de oro de la POO: los datos y las acciones que los manejan **viven juntos, dentro de la clase**. El stock y la acción de despachar están en el mismo lugar; el objeto se cuida solo.

Anatomía de un método (se escribe dentro de la clase):

```
void despachar(int cajas) {
    ...
}
```

| | | | |
|--------------------|---------------------------------|----------------------------|-------------|
| void | despachar | (int cajas) | { ... } |
| ↑ | ↑ | ↑ | ↑ |
| "no devuelve nada" | nombre de la acción (camelCase) | lo que RECIBE (parámetros) | lo que HACE |

`void` significa "esta acción solo hace cosas, no entrega ningún valor". Dentro del método, los atributos se tocan con `this`: `this.stockCajas` = "MI stock — el del objeto que está ejecutando la acción". Y desde afuera, la orden se da con el punto **y** **paréntesis**:

```
caramelos.despachar(12); // jorden dada! Los paréntesis son el "ejecútate"
```

 **Atributo sin paréntesis, método con paréntesis.** `caramelos.stockCajas` es un dato que consultas; `caramelos.despachar(12)` es una orden que das. Cuando la orden lleva datos (el 12), esos datos aterrizan en los parámetros del método.

 **Aquí brilla lo que ya sabes:** dentro de un método caben todas tus estructuras de control. Un `if` que valida, un `for` que repite, un `switch` que clasifica. Los métodos son el lugar donde tu conocimiento anterior se vuelve comportamiento de objetos.

■ EJEMPLO PRÁCTICO — Recibir y despachar con reglas

```
public class Main {
    public static void main(String[] args) {

        Producto caramelos = new Producto();
        caramelos.nombre = "Caramelos de café";
        caramelos.stockCajas = 50;

        caramelos.mostrarFicha();

        caramelos.recibir(30);           // llega producción
        caramelos.despachar(45);         // pedido grande: alcanza
        caramelos.despachar(100);        // pedido gigante: NO alcanza
        caramelos.despachar(-5);         // dato absurdo: rechazado

        caramelos.mostrarFicha();
    }
}

class Producto {
    String nombre;
    int stockCajas;

    void mostrarFicha() {
        System.out.println("🍬 " + this.nombre + " - stock: " + this.stockCajas + " cajas");
    }

    void recibir(int cajas) {
        if (cajas > 0) {
            this.stockCajas = this.stockCajas + cajas;
            System.out.println("⬆ Entraron " + cajas + " cajas de " + this.nombre);
        } else {
            System.out.println("🚫 Cantidad inválida");
        }
    }

    void despachar(int cajas) {
        if (cajas <= 0) {
            System.out.println("🚫 Cantidad inválida");
        } else if (cajas > this.stockCajas) {
            System.out.println("🚫 Stock insuficiente: hay " + this.stockCajas + " y piden " +
cajas);
        } else {
            this.stockCajas = this.stockCajas - cajas;
            System.out.println("⬇ Salieron " + cajas + " cajas de " + this.nombre);
        }
    }
}
```

■ En pantalla verás:

```
🍬 Caramelos de café - stock: 50 cajas
⬆ Entraron 30 cajas de Caramelos de café
⬇ Salieron 45 cajas de Caramelos de café
🚫 Stock insuficiente: hay 35 y piden 100
```

🚫 Cantidad inválida
 📦 Caramelos de café — stock: 35 cajas

Mira dónde quedaron las reglas: la validación del stock **vive dentro de `despachar`**, en la clase. Cualquier parte del programa que despache cualquier producto respeta las reglas **sin saberlo**. El stock en -12 de ayer acaba de volverse imposible. Eso es diseño, no suerte.

PRUEBA Y DESCUBRE (10 XP)

1. Escribe `caramelos.despachar`; sin paréntesis. El compilador se queja. Los paréntesis no son adorno: son la orden de ejecución.
2. Dentro de `recibir`, cambia `this.stockCajas` por `stockCajas` a secas. ¿Funciona? Sí — Java entiende el `this` implícito cuando no hay ambigüedad. En este taller **escribiremos `this` siempre**: gratis hoy, te salva la vida en el Nivel 4.
3. Crea un segundo producto (`galletas`, stock 10) y ordénale `galletas.despachar(45)`; . El mismo método, ejecutado por otro objeto, responde con **sus** números. Un método escrito una vez sirve para todos los objetos del molde.
4. Mueve el método `mostrarFicha` FUERA de la clase (déjalo suelto entre las dos clases). Lee el error del compilador: los métodos viven dentro de una clase, siempre.

HAZLO TÚ — El camión operativo (30 XP)

Retoma tu clase `Camion` del Nivel 1 y dale comportamiento:

1. `void mostrarFicha()` — imprime placa, ciudad, carga actual y capacidad en una línea clara.
2. `void cargar(double kg)` — suma a la carga **solo si**: los kg son positivos, el camión está disponible y la carga resultante no supera la capacidad. Cada rechazo con su mensaje específico (tres mensajes distintos: usa `if / else if / else if / else`).
3. `void descargar(double kg)` — resta con las validaciones equivalentes (nunca dejar la carga negativa).
4. `void enviarATaller()` — pone `disponible = false` y anuncia "🔧 Camión ABC-123 fuera de servicio".
5. En `main`: fabrica un camión de 1 000 kg de capacidad y ejecuta esta secuencia, verificando cada mensaje: cargar 400 → cargar 700 (debe rechazar por capacidad) → cargar 500 → descargar 200 → enviar a taller → cargar 100 (debe rechazar por no disponible) → ficha final. **Carga final correcta: 700.0 kg.**

Pistas: 🔑 1) "La carga resultante no supera la capacidad" se pregunta ANTES de sumar: `this.cargaActualKg + kg <= this.capacidadKg`. 🔑 2) El orden de los `else if` importa: valida primero lo más básico (kg positivos). 🔑 3) Contrasta tu salida con la secuencia esperada línea por línea.

RETO ADICIONAL (20 XP)

Agrega `void cargarVariosViajes(double kgPorViaje, int viajes)` que use un `for` para intentar cargar esa cantidad tantas veces como viajes, llamando por dentro a tu propio método `cargar` (sí: **un método puede llamar a otro con `this.cargar(...)`**). Pruébalo con 3 viajes de 300 kg en un camión de 1 000: el tercero debe rechazarse solo, sin que tú escribas ninguna validación nueva. Esa es la recompensa de encapsular las reglas: se reutilizan gratis.

DETECTIVE DE ERRORES #2 (20 XP)

El contador de despachos del turno no cuenta nada:

```
class Producto {
    String nombre;
    int stockCajas;
    int despachosDelDia;

    void despachar(int cajas) {
        int despachosDelDia = 0;
        if (cajas > 0 && cajas <= this.stockCajas) {
            this.stockCajas = this.stockCajas - cajas;
            despachosDelDia = despachosDelDia + 1;
        }
        System.out.println("Despachos del día: " + this.despachosDelDia);
    }
}
```

Diagnóstico: ① Tras 3 despachos válidos, ¿qué imprime siempre y por qué? ② La primera línea del método creó algo que **no** es el atributo, aunque se llame igual: ¿cómo se llama ese fenómeno y cuál de los dos `despachosDelDia` se está incrementando? (Pista: una variable local nueva en cada llamada eclipsa al atributo.) ③ Corrígelo borrando UNA línea y poniendo `this.` donde corresponde. ④ Moraleja para tu cuaderno: ¿por qué escribir `this.` siempre te habría salvado aquí?

USA LA IA PARA APRENDER (10 XP)

Modo revisor de código: pídele *"Escribe la clase `Bascula` para una planta con métodos `pesar` y `tarar`" (+ filtro)*. Audita como jefe de calidad: ¿validó pesos negativos? ¿Usó `this`? ¿Metió `return` o `private` que no hemos visto (recórtalos)? Escríbele tu veredicto en 2 líneas y observa si corrige bien. Bitácora.

 CHECKPOINT — XP: __/90

☐ Distingo atributo (sin paréntesis) de método (con paréntesis) · ☐ Escribo `this.` siempre y sé por qué · ☐ Mis validaciones viven DENTRO de los métodos · ☐ Sé que un método puede llamar a otro con `this.`

NIVEL 3 — PRODUCTOS QUE RESPONDEN

Métodos que devuelven valores (**return**)

 45 min · 90 XP

TE EXPLICO

Hay acciones que no imprimen: **calculan y entregan el resultado** para que otra parte del programa lo use. Para eso el método declara **qué tipo de dato devuelve** (en lugar de **void**) y lo entrega con **`return`**:

```
double pesoTotal() {
    return this.stockCajas * this.pesoCajaKg;
}
```

Desde afuera, el valor **llega** y se puede guardar, comparar o imprimir:

```
double kg = caramelos.pesoTotal();           // el resultado aterrizo en la variable
if (caramelos.pesoTotal() > 500) { ... }      // o se usa directo en una condición
```

 **¿Cuándo `void` y cuándo `return`?** Pregúntate: ¿esta acción produce un valor que alguien más necesita usar? **mostrarFicha()** solo actúa → **void**. **pesoTotal()** produce un número que se comparará o facturará → **return double**.

 **Imprimir NO es devolver.** Un método **void** con **println** muestra el dato y lo pierde; un método con **return** lo entrega vivo. Java es estricto con esto: si declaras **double** y no hay **return**, el compilador te frena con **missing return statement** — te está protegiendo de promesas incumplidas.

Tres tipos de retorno que usarás sin parar:

| Devuelve | Ejemplo de método | Se usa así |
|---------------------|---|---------------------------------------|
| double / int | pesoTotal() , cajasFaltantes(int meta) | en cálculos y comparaciones |
| boolean | puedeDespachar(int cajas) — nómbralo como pregunta | directo dentro de un if |
| String | ficha() — arma el texto y lo devuelve | System.out.println(p.ficha()); |

Y la jugada elegante del nivel: **un método que usa a otro**. **despachar** ya no repite la validación: **le pregunta a puedeDespachar**:

```
void despachar(int cajas) {
    if (this.puedeDespachar(cajas)) { ... }
}
```

EJEMPLO PRÁCTICO — El producto que responde preguntas

```
public class Main {
    public static void main(String[] args) {

        Producto arequipe = new Producto();
        arequipe.nombre = "Arequipe de café";
        arequipe.stockCajas = 40;
        arequipe.pesoCajaKg = 4.8;
        arequipe.precioCaja = 21000;

        System.out.println(arequipe.ficha());
        System.out.println("Peso total en bodega: " + arequipe.pesoTotal() + " kg");
        System.out.println("Valor del inventario: $" + arequipe.valorInventario());

        // Los boolean se leen como preguntas dentro del if
        if (arequipe.puedeDespachar(35)) {
            System.out.println("✅ El pedido de 35 cajas es viable");
        }

        arequipe.despachar(35);
        arequipe.despachar(20);    // ya no alcanza: el método lo sabe solo

        System.out.println(arequipe.ficha());
    }
}

class Producto {
    String nombre;
    int stockCajas;
    double pesoCajaKg;
    int precioCaja;

    double pesoTotal() {
        return this.stockCajas * this.pesoCajaKg;
    }

    int valorInventario() {
        return this.stockCajas * this.precioCaja;
    }

    boolean puedeDespachar(int cajas) {
        return cajas > 0 && cajas <= this.stockCajas;
    }

    void despachar(int cajas) {
        if (this.puedeDespachar(cajas)) {    // un método consultando a otro
            this.stockCajas = this.stockCajas - cajas;
            System.out.println("📦 Salieron " + cajas + " cajas de " + this.nombre);
        } else {
            System.out.println("🚫 Despacho de " + cajas + " cajas rechazado");
        }
    }

    String ficha() {
```

```

    return "📦 " + this.nombre + " | " + this.stockCajas + " cajas | "
           + this.pesoTotal() + " kg | $" + this.valorInventario();
}
}

```

🖥 En pantalla verás:

```

📦 Arequipe de café | 40 cajas | 192.0 kg | $840000
Peso total en bodega: 192.0 kg
Valor del inventario: $840000
✅ El pedido de 35 cajas es viable
📦 Salieron 35 cajas de Arequipe de café
🚫 Despacho de 20 cajas rechazado
📦 Arequipe de café | 5 cajas | 24.0 kg | $105000

```

Tres cosas para saborear: `puedeDespachar` devuelve la respuesta en una sola línea (la condición **ya es** un boolean — no necesita `if (cond) return true; else return false;`); `despachar` reutiliza esa pregunta con `this`. en vez de repetir la lógica; y `ficha()` llama por dentro a `pesoTotal()` y `valorInventario()` — **los métodos se arman unos con otros, como piezas de LEGO.**



PRUEBA Y DESCUBRE (10 XP)

1. En `pesoTotal`, borra el `return` y deja solo `this.stockCajas * this.pesoCajaKg`. Lee el error: `missing return statement`. Java no perdona promesas incumplidas.
2. Llama `arequipe.pesoTotal()`; en una línea sola, sin guardar ni imprimir. Compila y corre... y el valor **se calcula y se pierde**: nadie lo recibió. Moraleja sobre la diferencia entre ejecutar y aprovechar.
3. Cambia `ficha()` para que **imprima** en vez de devolver (tipo `void` con `println`). Ahora intenta `String f = arequipe.ficha()`; Lee el error de tipos. ¿Ves por qué devolver es más flexible que imprimir?
4. Curiosidad de precisión: `pesoTotal()` multiplica `int * double`. ¿De qué tipo sale? (`double`: Java "asciende" al tipo más amplio.) ¿Y `valorInventario()` con `int * int`? Verifica con las salidas.



HAZLO TÚ — El camión que responde (30 XP)

Evoluciona tu `Camion` con métodos que devuelven:

1. `double espacioLibre()` — capacidad menos carga actual.
2. `boolean cabe(double kg)` — pregunta con nombre de pregunta: ¿positivo, disponible y con espacio? (Devuelve la condición directa, sin `if`.)
3. `double porcentajeOcupacion()` — carga ÷ capacidad × 100.
4. `String ficha()` — devuelve la ficha completa usando por dentro `espacioLibre()` y `porcentajeOcupacion()`.

5. **Reescribe** `cargar(double kg)` para que consulte `this.cabe(kg)` en vez de repetir condiciones.
6. En `main`: camión de 1 000 kg con 350 ya cargados → imprime la ficha, pregunta con un `if` si caben 700 (no deben caber), carga 500, y cierra con la ficha final. **Verifica: ocupación final 85.0%.**

Pistas: 🔑 1) Cada método nuevo debería tener 1–3 líneas: si te está quedando largo, estás repitiendo lógica que ya vive en otro método. 🔑 2) El porcentaje necesita `double`; si la capacidad fuera `int`, cuidado con la división entera. 🔑 3) `ficha()` **devuelve**: el `println` va en `main`.

🚀 RETO ADICIONAL (20 XP)

`int viajesNecesarios(double kgTotales)` — cuántos viajes completos necesita el camión para mover esa carga, usando su capacidad. Pista matemática con lo que ya sabes: divide, y si queda residuo, suma un viaje (un `if` con el operador `%` no sirve aquí porque son `double`; compara `viajes * capacidad < kgTotales`). Pruébalo con 2 500 kg en un camión de 1 000: **3 viajes**.

🐛 DETECTIVE DE ERRORES #3 (20 XP)

El reporte financiero de la planta compila... a medias, y lo que compila, miente:

```
class Producto {
    String nombre;
    int stockCajas;
    int precioCaja;

    int valorInventario() {
        System.out.println(this.stockCajas * this.precioCaja);
    }

    boolean esValioso() {
        if (this.valorInventario() > 500000) {
            return true;
        }
    }
}
```

Diagnóstico: ① `valorInventario` promete un `int` — ¿qué hace en su lugar y qué dice el compilador? ② `esValioso` tiene un agujero lógico que Java detecta: ¿qué devuelve cuando la condición es falsa? (Nada — y `missing return statement` otra vez.) ③ Reescribe ambos en su forma mínima: el primero con `return`, el segundo en **una sola línea sin `if`**. ④ Reflexión: ¿por qué es una suerte que Java se niegue a compilar esto, en vez de dejarlo correr?



USA LA IA PARA APRENDER (10 XP)

El duelo de la línea única: pídele "Escribe en Java un método boolean `esRentable()` que devuelva si `preciostock` supera 500000, de la forma más corta posible" (+ filtro). Compara con TU versión. Luego pregúntale: "¿por qué `return condicion;` es mejor que `if (condicion) return true; else return false;`?" y evalúa si su explicación menciona que la condición ya ES un boolean*. Bitácora.



CHECKPOINT — XP: ____/90

☐ Sé cuándo `void` y cuándo `return` · ☐ Devuelvo booleanos con la condición directa · ☐ Armo métodos que usan otros métodos con `this.` · ☐ Entendí que imprimir no es devolver

NIVEL 4 — LA MÁQUINA DE NACER 🐣

El constructor y `this`

🕒 45 min · 90 XP

🎓 TE EXPLICO

Fabricar un producto hoy toma cinco líneas y ninguna garantía: si olvidas asignar el nombre, tienes un producto `null` deambulando por la bodega (lo viste en el Nivel 1). Queremos poder escribir:

```
Producto caramelos = new Producto("Caramelos de café", 6.2, 18500);
```

Una línea. Todos los datos. Imposible olvidar algo. Eso lo logra el **constructor**:

El constructor es la sala de partos de la clase: un bloque que se ejecuta automáticamente, una vez, en el instante exacto del `new`. Ahí exiges los datos de nacimiento y dejas al objeto completo y válido desde su primera línea de vida.

```
class Producto {
    String nombre;
    double pesoCajaKg;
    int precioCaja;
    int stockCajas;

    Producto(String nombre, double pesoCajaKg, int precioCaja) {
        this.nombre = nombre;
        this.pesoCajaKg = pesoCajaKg;
        this.precioCaja = precioCaja;
        this.stockCajas = 0; // todo producto nace sin stock
        System.out.println("📦 Producto registrado: " + this.nombre);
    }
}
```

Las tres reglas del constructor en Java:

1. Se llama **exactamente igual que la clase**, con su misma mayúscula: clase `Producto` → constructor `Producto(...)`.
2. **No declara tipo de retorno**. Ni `void`, ni nada. `Producto(...)` a secas. (Si le pones `void`, deja de ser constructor — lo verás en el Detective de hoy.)
3. Decide qué se **exige** como parámetro (lo que varía entre productos: nombre, peso, precio) y qué se **fija** adentro (lo que todos comparten al nacer: stock 0).

Y ahora sí, `this` en su máximo esplendor. La línea `this.nombre = nombre;` desconcierta la primera vez:

```

this.nombre      =      nombre
  ↑                  ↑
"MI atributo nombre"  "el dato que me acaban
(el del objeto que   de entregar" (el parámetro)
está naciendo)

```

Como el parámetro y el atributo se llaman igual (costumbre profesional), `this.` es lo único que los distingue. Sin él, `nombre = nombre` asigna el parámetro a sí mismo y el atributo queda `null` — el bug de constructores más famoso del planeta, y hoy lo vas a cazar.

💡 ¿Y qué pasó con `new Producto()` vacío? Mientras tu clase no tenía constructor, Java te regalaba uno invisible sin parámetros. En cuanto escribes el tuyo, el regalo desaparece: `new Producto()` a secas dejará de compilar. Es a propósito — tú decidiste que nadie nace sin datos.

📁 EJEMPLO PRÁCTICO — El registro oficial de productos

```

public class Main {
    public static void main(String[] args) {

        Producto caramelos = new Producto("Caramelos de café", 6.2, 18500);
        Producto arequipe = new Producto("Arequipe de café", 4.8, 21000);
        Producto galletas = new Producto("Galletas de café", -3.0, 15000); // ¡peso absurdo!

        caramelos.recibir(50);
        arequipe.recibir(30);

        System.out.println(caramelos.ficha());
        System.out.println(arequipe.ficha());
        System.out.println(galletas.ficha());
    }
}

class Producto {
    String nombre;
    double pesoCajaKg;
    int precioCaja;
    int stockCajas;

    Producto(String nombre, double pesoCajaKg, int precioCaja) {
        this.nombre = nombre;
        this.precioCaja = precioCaja;
        this.stockCajas = 0;

        if (pesoCajaKg > 0) { // el constructor también DEFIENDE
            this.pesoCajaKg = pesoCajaKg;
        } else {
            this.pesoCajaKg = 1.0;
            System.out.println("⚠️ Peso inválido para " + nombre + ": se ajustó a 1.0 kg");
        }

        System.out.println("📦 Producto registrado: " + this.nombre);
    }
}

```

```

void recibir(int cajas) {
    if (cajas > 0) {
        this.stockCajas = this.stockCajas + cajas;
    }
}

String ficha() {
    return "📦 " + this.nombre + " | " + this.stockCajas + " cajas | "
        + this.pesoCajaKg + " kg/caja | $" + this.precioCaja;
}
}

```

🖥 En pantalla verás:

```

📦 Producto registrado: Caramelos de café
📦 Producto registrado: Arequipe de café
⚠️ Peso inválido para Galletas de café: se ajustó a 1.0 kg
📦 Producto registrado: Galletas de café
📦 Caramelos de café | 50 cajas | 6.2 kg/caja | $18500
📦 Arequipe de café | 30 cajas | 4.8 kg/caja | $21000
📦 Galletas de café | 0 cajas | 1.0 kg/caja | $15000

```

Fíjate: el mensaje "registrado" salió tres veces **sin que nadie lo llamara** — el constructor corre solo, una vez por `new`. Y el peso absurdo de las galletas fue **interceptado en el nacimiento**: el constructor no solo llena datos, **defiende al objeto desde su primer segundo**. Un objeto que nace válido no necesita que lo arreglen después.



PRUEBA Y DESCUBRE (10 XP)

1. Intenta `new Producto()`; vacío. Lee el error: el constructor regalado desapareció cuando escribiste el tuyo. Ahora nacer exige datos.
2. Intenta `new Producto("Chocolatinas")`; con un solo dato. El compilador cuenta los parámetros por ti. ¿Qué dice?
3. El clásico: dentro del constructor cambia `this.nombre = nombre;` por `nombre = nombre;`. Compila y corre... pero imprime la ficha. ¿Qué salió en el nombre y por qué? (El aviso amarillo del compilador —si tu entorno lo muestra— dice algo como *"assignment to itself"*.)
4. Agrega `System.out.println(this);` al final del constructor y crea dos productos. Salen dos códigos raros distintos (`Producto@1b6d...`): son las **direcciones de memoria** — la prueba visual de que cada `new` fabrica un objeto distinto y `this` apunta al recién nacido de turno.



HAZLO TÚ — La flota nace completa (30 XP)

Rediseña `Camion` con constructor:

1. Al fabricar un camión **se exigen**: placa, capacidad en kg y ciudad.
2. Todo camión **nace** con carga 0.0 y disponible en **true** (fijados adentro, no pedidos).
3. El constructor **defiende**: si la capacidad recibida es menor que 100, la ajusta a 100 y avisa (ningún camión de juguete en esta planta).
4. Anuncia cada nacimiento: " Camión ABC-123 asignado a Armenia (1200.0 kg)".
5. Conserva tus métodos de los Niveles 2–3 (**cargar**, **cabe**, **espacioLibre**, **ficha**) — deben seguir funcionando sin tocarlos.
6. En **main**: fabrica tres camiones en tres líneas (uno con capacidad inválida, p. ej. 50), carga algo en cada uno e imprime las tres fichas.

Pistas:  1) Decide con la regla del nivel: ¿qué varía entre camiones? → parámetros. ¿Qué comparten todos al nacer? → se fija adentro.  2) La defensa de la capacidad es un **if/else** dentro del constructor, como el peso de las galletas.  3) Si algún método viejo deja de compilar, revisa que no dependiera de asignar atributos desde **main**.

RETO ADICIONAL (20 XP)

Agrega a **Producto** el atributo **int totalDespachado** (nace en 0) y haz que **despachar** lo acumule. Luego un método **String resumenDelDia()** que devuelva stock actual + total despachado. Todo el seguimiento del día quedó **dentro** del objeto: nadie afuera lleva cuentas paralelas. Pruébalo con tres despachos.

DETECTIVE DE ERRORES #4 — el más traicionero del taller (20 XP)

Este código **compila perfectamente**. Y aun así, todo sale mal:

```
public class Main {
    public static void main(String[] args) {
        Cliente c1 = new Cliente();
        System.out.println(c1.nombre + " | pedidos: " + c1.pedidos);
    }
}

class Cliente {
    String nombre;
    int pedidos;

    void Cliente(String nombre) {
        this.nombre = nombre;
        this.pedidos = 1;
        System.out.println("
```

Diagnóstico: ① Ejecuta. ¿Qué imprime la ficha y qué mensaje NUNCA apareció? ② La palabra `void` convirtió al "constructor" en... ¿qué exactamente? ¿Por qué entonces `new Cliente()` sí compiló? (Pista: como la clase quedó "sin constructor", Java volvió a regalar el vacío.) ③ ¿Por qué este bug es más peligroso que uno que no compila? ④ Corrígelo (quita una palabra) y verifica que ahora `new Cliente()` vacío **falle** y `new Cliente("Dulcería El Trébol")` funcione.

USA LA IA PARA APRENDER (10 XP)

La IA casi siempre escribe constructores con cosas que no hemos visto. Pídele "Crea la clase `Pedido` en Java con constructor" **sin** el filtro, a propósito. Identifica todo lo prohibido que trajo (`private`, getters, `toString`, sobrecarga...) y **tradúcela** a la versión de este taller: atributos simples + constructor + `this`. Luego repite **CON** el filtro y compara: ¿obedeció? Este ejercicio vale doble en la bitácora: documenta la traducción.

CHECKPOINT — XP: ____/90

☐ Mi constructor se llama como la clase y no lleva tipo de retorno · ☐ Distingo qué se exige y qué se fija al nacer · ☐ `this.nombre = nombre` ya no tiene misterio · ☐ Sé que un constructor también valida y defiende

NIVEL 5 — LA PLANTA EN MOVIMIENTO

Objetos que colaboran entre sí

 50 min · 90 XP

TE EXPLICO

Hasta aquí cada objeto vivía solo. Pero una planta real es una **conversación entre objetos**: el camión carga *productos*, el pedido pertenece a *un cliente*, la bodega compara *dos lotes*. La pieza que faltaba es simple y poderosa:

Un método puede recibir OTRO OBJETO como parámetro — y entonces puede leerle sus atributos y darle órdenes con sus métodos.

```
void cargarProducto(Producto p, int cajas) {
    double kilos = cajas * p.pesoCajaKg;    // Le LEO un atributo al producto
    ...
    p.despachar(cajas);                    // y Le doy una ORDEN
}
```

Fíjate en el parámetro: **Producto p**. Tu clase es un tipo, igual que **int** o **String** — así que puede viajar por parámetros. Dentro del método, **p** es el producto que te entregaron, y **this** sigue siendo el objeto que ejecuta (el camión). Dos objetos, cada uno con su nombre, conversando en la misma línea.

Con esto se abren tres jugadas profesionales:

1. **Colaboración**: el camión descuenta del producto y se carga a sí mismo, cada quien validando lo suyo.
2. **Comparación**: un método que recibe otro objeto del mismo tipo — **boolean tieneMasStockQue(Producto otro)** — y compara **this.stockCajas > otro.stockCajas**.
3. **Objetos dentro de objetos**: un atributo también puede ser de tipo clase — **Cliente cliente**; dentro de **Pedido**. La ficha del pedido puede entonces decir **this.cliente.nombre**: el punto se encadena, y se lee igual que siempre: *"el nombre del cliente de este pedido"*.

⚠ Una advertencia con nombre propio: si un atributo-objeto nunca se asigna, vale **null** (Nivel 1). Pedirle algo a **null** — **this.cliente.nombre** cuando no hay cliente — revienta el programa **en plena ejecución** con la excepción más famosa de Java: **NullPointerException**. La regla defensiva: antes de usar un objeto que podría faltar, pregunta **if (this.cliente != null)**.

EJEMPLO PRÁCTICO — El despacho conversado

```

public class Main {
    public static void main(String[] args) {

        Producto caramelos = new Producto("Caramelos de café", 6.2);
        Producto arequipe = new Producto("Arequipe de café", 4.8);
        caramelos.recibir(50);
        arequipe.recibir(80);

        Camion camion = new Camion("ABC-123", 300.0);

        // El camión conversa con los productos
        camion.cargarProducto(caramelos, 20);    // 124 kg: cabe
        camion.cargarProducto(arequipe, 30);    // 144 kg: cabe justo
        camion.cargarProducto(arequipe, 10);    // 48 kg más: YA NO cabe

        System.out.println(camion.ficha());

        // Los productos se comparan entre sí
        if (caramelos.tieneMasStockQue(arequipe)) {
            System.out.println("🏆 Más stock restante: " + caramelos.nombre);
        } else {
            System.out.println("🏆 Más stock restante: " + arequipe.nombre);
        }
    }
}

class Producto {
    String nombre;
    double pesoCajaKg;
    int stockCajas;

    Producto(String nombre, double pesoCajaKg) {
        this.nombre = nombre;
        this.pesoCajaKg = pesoCajaKg;
        this.stockCajas = 0;
    }

    void recibir(int cajas) {
        if (cajas > 0) { this.stockCajas = this.stockCajas + cajas; }
    }

    boolean puedeDespachar(int cajas) {
        return cajas > 0 && cajas <= this.stockCajas;
    }

    void despachar(int cajas) {
        if (this.puedeDespachar(cajas)) { this.stockCajas = this.stockCajas - cajas; }
    }

    boolean tieneMasStockQue(Producto otro) {
        return this.stockCajas > otro.stockCajas;    // yo vs. el otro
    }
}

class Camion {
    String placa;
    double capacidadKg;
    double cargaActualKg;

    Camion(String placa, double capacidadKg) {
        this.placa = placa;
        this.capacidadKg = capacidadKg;
        this.cargaActualKg = 0.0;
    }
}

```

```

void cargarProducto(Producto p, int cajas) {
    double kilos = cajas * p.pesoCajaKg;

    if (!p.puedeDespachar(cajas)) {
        System.out.println("🚫 " + p.nombre + " no tiene " + cajas + " cajas");
    } else if (this.cargaActualKg + kilos > this.capacidadKg) {
        System.out.println("🚫 No caben " + kilos + " kg en el camión " + this.placa);
    } else {
        p.despachar(cajas); // orden al producto
        this.cargaActualKg = this.cargaActualKg + kilos; // actualización propia
        System.out.println("✅ " + cajas + " cajas de " + p.nombre + " (" + kilos + " kg) al camión " + this.placa);
    }
}

String ficha() {
    return "🚚 " + this.placa + " | carga: " + this.cargaActualKg + " / " + this.capacidadKg + " kg";
}

```

🖥 En pantalla verás:

```

✅ 20 cajas de Caramelos de café (124.0 kg) al camión ABC-123
✅ 30 cajas de Arequipe de café (144.0 kg) al camión ABC-123
🚫 No caben 48.0 kg en el camión ABC-123
🚚 ABC-123 | carga: 268.0 / 300.0 kg
🏆 Más stock restante: Arequipe de café

```

Léelo como arquitecto: en `cargarProducto` conviven **dos validaciones de dos dueños distintos** — el producto responde por su stock (`p.puedeDespachar`), el camión responde por su capacidad (`this.cargaActualKg + kilos`). Ninguno invade las reglas del otro, y solo si ambos dicen sí, la operación ocurre en los dos a la vez. Así se diseña software real: **cada objeto es responsable de lo suyo, y los métodos son el idioma en que negocian.**

🔍 PRUEBA Y DESCUBRE (10 XP)

1. En `tieneMasStockQue`, llama `caramelos.tieneMasStockQue(caramelos)` — un producto comparado consigo mismo. ¿Qué devuelve y por qué tiene sentido?
2. Declara `Camion c2;` sin `new` y llama `c2.ficha();`... el compilador te frena antes. Ahora fuérralo: `Camion c2 = null; System.out.println(c2.ficha());`. Compila... y en ejecución: `NullPointerException`. Léela la línea exacta que señala. Acabas de conocer en carne propia al error #1 de Java.
3. Dentro de `cargarProducto`, imprime `this.placa` y `p.nombre` en la misma línea. Dos objetos distintos, cada uno con su nombre, visibles a la vez: `this` es quien ejecuta, `p` es quien visita.
4. ¿Qué pasa si llamas `camion.cargarProducto(caramelos, -5)`? Síguelo con la mirada: ¿cuál de las dos validaciones lo atrapa, la del producto o la del camión?

🔨 HAZLO TÚ — El pedido completo (30 XP)

Modela la pieza que une todo:

1. Clase `Cliente`: nombre y ciudad, ambos exigidos por constructor.
2. Clase `Pedido` con atributos: `Cliente cliente` (¡un objeto adentro de otro!), `Producto producto`, `int cajas` y `boolean despachado` (nace `false`). El constructor exige cliente, producto y cajas.
3. Método `double pesoDelPedido()` — cajas × peso por caja **del producto** (punto encadenado).
4. Método `void procesar(Camion camion)` — recibe el camión, y **solo si** el pedido no está ya despachado, le ordena `camion.cargarProducto(this.producto, this.cajas)`; luego marca `despachado = true` únicamente si el producto realmente bajó de stock (pista: guarda el stock antes y compáralo después, o pregunta primero `producto.puedeDespachar` y la capacidad — elige tu estrategia y coméntala).
5. Método `String ficha()` — devuelve algo como `"📄 Pedido de Dulcería El Trébol (Armenia): 15 cajas de Arequipe de café – 72.0 kg – despachado: true"`.
6. En `main`: un cliente, dos productos con stock, un camión de 500 kg, y **tres pedidos**: uno normal, uno que exceda el stock y uno que intente procesarse dos veces. Imprime las tres fichas finales.

Pistas: 🔑 1) El punto se encadena con calma: `this.producto.pesoCajaKg`. 🔑 2) "Procesarse dos veces" se bloquea con `if (this.despachado)` al inicio del método. 🔑 3) Si algo revienta con `NullPointerException`, un objeto no se fabricó: busca el `new` que falta.

🚀 RETO ADICIONAL (20 XP)

`boolean mismoDestino(Pedido otro)` en la clase `Pedido`: devuelve si dos pedidos van a la misma ciudad. Ojo doble: la ciudad vive en `this.cliente.ciudad` y `otro.cliente.ciudad`... y son `String`, así que la comparación correcta es con `.equals(...)`, no con `==` (con `==` Java compara direcciones de memoria, no letras — pruébalo con dos clientes distintos de la misma ciudad y mira cómo `==` te miente). Documenta el experimento.

🐛 DETECTIVE DE ERRORES #5 (20 XP)

El "duplicador de camiones" del turno de la noche:

```
Camion original = new Camion("ABC-123", 1000.0);
original.cargaActualKg = 400.0;

Camion respaldo = original;
respaldo.placa = "XYZ-999";
respaldo.cargaActualKg = 0.0;
```

```
System.out.println("Original: " + original.placa + " con " + original.cargaActualKg + " kg");
System.out.println("Respaldo: " + respaldo.placa + " con " + respaldo.cargaActualKg + " kg");
```

Diagnóstico: ① Predice la salida ANTES de ejecutar. Luego ejecuta. ② ¿Cuántos objetos `Camion` existen realmente en memoria? Dibuja las dos variables y sus flechas. ③ ¿Por qué "vaciar el respaldo" vació también el original? ④ Escribe el duplicado correcto (pista: un segundo `new` y copiar atributo por atributo). ⑤ Conecta: este es el experimento 3 del Nivel 1, ahora costándole 400 kg fantasma a la planta.

USA LA IA PARA APRENDER (10 XP)

El examen final invertido: pídele *"Hazme 5 preguntas conceptuales sobre clases, atributos, métodos y constructores en Java, una por una, y califícame del 1 al 10 cada respuesta"*. Respóndelas **sin mirar el taller**. Anota tu nota global y tu punto más débil en la bitácora — ese punto es lo primero que repasas antes del jefe final.

CHECKPOINT — XP: ____/90

☐ Paso objetos como parámetros y encadeno el punto · ☐ Sé quién es `this` y quién es el objeto visitante · ☐ Reconozco un `NullPointerException` y sé buscar el `new` que falta · ☐ Comparo `String` con `.equals`, no con `==`

👑 JEFE FINAL — "BODEGA MAESTRA" 🏭

Proyecto integrador · 🕒 60–90 min · 150 XP

La situación

La jefa entra con el portátil bajo el brazo:

"Auditoría viene el lunes. Quiero UN programa que administre la bodega completa: productos que nacen bien, camiones que cargan con reglas y un menú para operarlo todo. Y me lo sustentas tú, línea por línea — ya sabes cómo funciona esto."

El sistema SIBOM (Sistema de Bodega — Misión Objetos)

Un solo archivo con **cuatro clases**: `Main`, `Producto`, `Camion` y `Pedido` (con su `Cliente` si seguiste el Nivel 5 completo — recomendado).

Arquitectura obligatoria:

1. ``Producto`` — atributos: nombre, peso por caja, precio por caja, stock, total despachado. Constructor que exige nombre, peso y precio (stock y total nacen en 0) y **defiende** contra peso o precio no positivos. Métodos: `recibir`, `puedeDespachar`, `despachar` (que acumula el total), `pesoTotal`, `valorInventario` y `ficha` (que reutiliza los anteriores).
2. ``Camion`` — constructor que exige placa y capacidad (carga nace en 0, disponible en `true`) con defensa de capacidad mínima. Métodos: `cabe`, `espacioLibre`, `cargarProducto(Producto p, int cajas)` con la doble validación del Nivel 5, y `ficha`.
3. ``Pedido`` (*nivel pro, opcional pero premiado*) — une cliente, producto y cajas; `procesar(Camion c)` y `ficha`.
4. ``Main`` — fabrica 3 productos y 2 camiones al inicio, y opera con un **menú** ``do-while`` + ``switch`` (¡tus estructuras de control, brillando!):

```
===== SIBOM — Dulce Café S.A. =====
1. Ver catálogo (fichas de los 3 productos)
2. Recibir producción (elige producto y cantidad)
3. Cargar camión (elige producto, camión y cajas)
4. Ver flota (fichas de los camiones)
5. Reporte general (valor total del inventario sumando los 3)
6. Cerrar bodega
```

Para "elegir producto", usa un `switch` sobre un número leído con `Scanner` (1 = caramelos, 2 = arequipe, 3 = galletas) — sin listas ni arreglos: tres variables de tipo `Producto` bastan.

Restricciones de calidad (aquí vive la nota)

✓ Solo los temas del taller: nada de `private`, herencia, `ArrayList` ni `static` extra. · ✓ Ninguna validación en ``main``: todas las reglas viven dentro de los métodos y constructores de las clases. · ✓ Ningún `String` comparado con `==`. · ✓ `this.` en todos los accesos a atributos propios. · ✓ Probado con **tres corridas**: una operación normal completa, una llena de datos inválidos (cantidades negativas, cargas imposibles) y una que intente romperlo con creatividad.

Entregables

| Entregable | XP |
|--|----|
| El programa corriendo con sus tres corridas de prueba | 60 |
| Diagrama a mano de tus clases: cada una con sus atributos y métodos, y flechas donde un método recibe un objeto de otra clase | 30 |
| Lista de 5 decisiones de diseño con su porqué ("el stock no es parámetro del constructor porque...") | 30 |
| Sustentación de 3 minutos a la jefa, sin tecnicismos sin traducir | 20 |
| Bitácora de IA completa de los 5 niveles | 10 |

🚩 **La regla de oro:** IA permitida para todo. Pero en la sustentación, cualquier línea señalada que no puedas explicar ("¿por qué está aquí y qué pasa si la borro?") vale 0. Es el simulacro exacto de tu primer empleo.

ANEXOS

ANEXO A — Chuleta: POO en Java en una página

```
// ---- CLASE Y ATRIBUTOS (el molde) ----
class Producto {
    String nombre;           // PascalCase
    int stockCajas;          // atributos en camelCase
    double pesoCajaKg;       // defectos de fábrica: 0, 0.0, false, null

    // ---- CONSTRUCTOR (La sala de partos) ----
    Producto(String nombre, double pesoCajaKg) { // mismo nombre de la clase, SIN tipo de retorno
        this.nombre = nombre; // this.atributo = parámetro
        this.pesoCajaKg = pesoCajaKg;
        this.stockCajas = 0; // Lo que todos comparten al nacer, se fija adentro
        if (pesoCajaKg <= 0) { this.pesoCajaKg = 1.0; } // el constructor también defiende
    }

    // ---- MÉTODOS ----
    void recibir(int cajas) { // void: solo actúa
        if (cajas > 0) { this.stockCajas += cajas; }
    }
    double pesoTotal() { // con return: calcula y ENTREGA
        return this.stockCajas * this.pesoCajaKg;
    }
    boolean puedeDespachar(int cajas) { // boolean: La condición YA es la respuesta
        return cajas > 0 && cajas <= this.stockCajas;
    }
    void despachar(int cajas) {
        if (this.puedeDespachar(cajas)) { // un método usa a otro con this.
            this.stockCajas -= cajas;
        }
    }
    String ficha() { // String: arma el texto y lo devuelve
        return this.nombre + " | " + this.stockCajas + " cajas | " + this.pesoTotal() + " kg";
    }
    boolean tieneMasStockQue(Producto otro) { // recibe OTRO objeto y lo compara
        return this.stockCajas > otro.stockCajas;
    }
}

// ---- OBJETOS (Las galletas del molde) ----
Producto p1 = new Producto("Caramelos", 6.2); // new fabrica; sin new no hay objeto
p1.recibir(50); // método: CON paréntesis
System.out.println(p1.ficha()); // el println va afuera; ficha devuelve
int s = p1.stockCajas; // atributo: SIN paréntesis
Producto p2 = p1; // ⚠ NO copia: dos nombres, UN objeto
// String se compara con .equals, nunca con ==
```

ANEXO B — Diccionario del compilador (versión objetos)

| Mensaje | Qué significa de verdad | Qué revisar primero |
|--------------------|--------------------------------|--|
| cannot find symbol | Ese nombre no existe para Java | Letra por letra y mayúsculas: nombre ≠ Nombre ; ¿el atributo está en el molde? |

| Mensaje | Qué significa de verdad | Qué revisar primero |
|---|---|---|
| <code>variable p might not have been initialized</code> | Declaraste sin fabricar | Falta el <code>new</code> |
| <code>constructor Producto in class Producto cannot be applied...</code> | Los datos del <code>new</code> no coinciden con el constructor | Cuenta y compara los parámetros |
| <code>missing return statement</code> | El método promete un tipo y hay caminos sin <code>return</code> | ¿Todos los <code>if/else</code> devuelven algo? |
| <code>incompatible types: void cannot be converted to String</code> | Guardaste el "resultado" de un método que no devuelve | El método imprime en vez de devolver |
| <code>';' expected / '}' expected</code> | Puntuación rota | Casi siempre en la línea ANTERIOR; cuenta llaves |
| <code>class X is public, should be declared in a file named X.java</code> | Regla del compilador en línea | Solo <code>Main</code> lleva <code>public</code> en un archivo único |
| <code>NullPointerException` (en ejecución)</code> | Le pediste algo a la nada | Un objeto quedó en <code>null</code> : busca el <code>new</code> que falta o el atributo-objeto sin asignar |
| El <code>if</code> con Strings "nunca entra" (sin error) | Comparaste texto con <code>==</code> | Usa <code>.equals(...)</code> |

🔑 Lee siempre el **primer** error de la lista y su **número de línea**; los demás suelen ser eco del primero. Arregla uno, recompila, repite.

ANEXO C — Soluciones de los Detectives

🔴 Intenta 10 minutos, usa las pistas, intenta 5 más. Solo entonces mira.

N1 · #1: ① `class cliente` debe ser `Cliente` (PascalCase): compila, pero rompe la convención que todo el mundo espera leer. ② El tipo es `String`, con mayúscula. ③ Falta `new`: `Cliente c1 = new Cliente();` — sin él, Java intenta llamar un método llamado `cliente()` que no existe. ④ `c1.Nombre` ≠ `c1.nombre`: Java distingue mayúsculas en TODO.

N2 · #2: ① Imprime siempre `0`. ② La línea `int despachosDelDia = 0;` crea una **variable local** que nace y muere en cada llamada, y **eclipse** al atributo (sombreado o *shadowing*); el incremento cae sobre la local, que se evapora. ③ Borrar esa línea y usar `this.despachosDelDia = this.despachosDelDia + 1;`. ④ Con `this.` en todas partes, la local jamás habría interceptado nada — por eso el taller lo exige.

N3 · #3: ① Imprime en vez de devolver → `missing return statement`. Corrección: `return this.stockCajas * this.precioCaja;`. ② Cuando la condición es falsa no devuelve nada → mismo error. ③ Forma mínima: `boolean`

`esValioso() { return this.valorInventario() > 500000; }.` ④ Es una suerte porque el error se descubre en compilación, gratis, y no en producción mintiendo.

N4 · #4: ① La ficha imprime `null | pedidos: 0` y el mensaje 🚗 jamás sale. ② `void Cliente(...)` es un **método común** que casualmente se llama como la clase; como la clase quedó "sin constructor", Java regaló el vacío y `new Cliente()` compiló usando ese regalo — el método nunca se ejecutó porque nadie lo llamó. ③ Es más peligroso porque **no hay ningún error**: el programa corre con datos de fábrica y nadie se entera hasta que la ficha sale vacía. ④ Quitar `void`. Con el constructor real, `new Cliente()` vacío deja de compilar (el regalo desaparece) y `new Cliente("Dulcería El Trébol")` funciona.

N5 · #5: ① Imprime `Original: XYZ-999 con 0.0 kg` y `Respaldo: XYZ-999 con 0.0 kg`. ② **Un solo objeto**: `respaldo = original` copió la dirección, no el camión; las dos flechas apuntan al mismo cuadro. ③ Porque "el respaldo" ES el original con otro nombre. ④ `Camion respaldo = new Camion(original.placa, original.capacidadKg); respaldo.cargaActualKg = original.cargaActualKg;` (y luego sí, cambiar placa). ⑤ Mismo fenómeno del Nivel 1, ahora con consecuencias de bodega.

ANEXO D — Rúbrica de evaluación

| Criterio | Excelente (5.0) | Bueno (4.0) | Aceptable (3.0) | Insuficiente (≤2.5) |
|--------------------------------------|--|---|--|---|
| Modelado (clases y atributos) | Clases con nombres y tipos que representan fielmente la planta | Modelado correcto con detalles | Clases funcionales con tipos débiles | Las clases no modelan el problema |
| Métodos | Distingue <code>void/return</code> con criterio; métodos cortos que se reutilizan entre sí; validaciones adentro | Métodos correctos con algo de repetición | Funcionan pero con lógica en <code>main</code> | Métodos que imprimen lo que debían devolver o sin validar |
| Constructor | Objetos nacen completos y defendidos; decide bien qué se exige y qué se fija | Constructores correctos | Funcionales pero permiten estados raros | <code>this</code> mal usado u objetos incompletos |
| Colaboración entre objetos | Objetos como parámetros con responsabilidades claras; sin aliasing accidental; <code>.equals</code> en Strings | Colaboración correcta con detalles | Colaboración con ayuda | Un solo objeto gigante o <code>==</code> en Strings |
| Robustez | Las tres corridas de prueba pasan; ningún <code>NullPointerException</code> sin resolver | Corridas normales bien, bordes con detalles | Solo la corrida feliz | Cualquier dato raro lo rompe |
| Uso de IA y sustentación | Bitácora completa, tradujo código avanzado de la IA, explica cada línea | Bitácora completa, uso pertinente | Bitácora incompleta | Copió sin comprender |

Nota final = promedio de los 6 criterios.

ANEXO E — Tablero de XP y Bitácora de IA

| Nivel |  (10) |  (30) |  (20) |  (20) |  (10) | Total /90 |
|--|--|--|--|--|--|-------------|
| 1 Clases y objetos | | | | | | |
| 2 Métodos void | | | | | | |
| 3 Métodos return | | | | | | |
| 4 Constructor | | | | | | |
| 5 Colaboración | | | | | | |
|  Bodega maestra | — | — | — | — | — | /150 |
| | | | | | GRAN TOTAL | /600 |

Insignias:  ☐ ·  ☐ ·  ☐ ·  ☐

Bitácora de IA (una fila por consulta):

| # | Nivel | Qué pedí | Qué respondió (resumen) | ¿Trajo temas prohibidos? | Qué corregí yo |
|---|-------|----------|-------------------------|---|----------------|
| 1 | | | | ☐ Sí ☐ No | |
| 2 | | | | ☐ Sí ☐ No | |
| 3 | | | | ☐ Sí ☐ No | |
| 4 | | | | ☐ Sí ☐ No | |
| 5 | | | | ☐ Sí ☐ No | |

Reflexión final (5 líneas): ¿Qué concepto te costó más y qué metáfora lo destrabó? ¿Cuántas veces la IA te ofreció código por encima del nivel y qué hiciste con él?